

BAGIAN 5 DARI 8

Sesi 3 — ETL dengan Python

Sesi ini mengerjakan ulang pekerjaan Sesi 1, kali ini dengan Python. Satu berkas, `sesi3_etl_python/etl.py`, dengan tiga bagian yang diberi garis pemisah mencolok di dalam kode:

```
# BAGIAN 1  EXTRACT    tarik apa adanya dari MySQL dan dari CSV
# BAGIAN 2  TRANSFORM  bersihkan, seragamkan, catat yang dibuang
# BAGIAN 3  LOAD       muat dimensi dulu, baru fakta, terakhir indeks
```

Di paket 8 sesi ini terpecah jadi dua sesi dan tiga berkas. Di paket inti ketiganya dalam satu berkas yang dijalankan sekali — satu perintah, satu laporan, selesai dalam hitungan detik.

```
python sesi3_etl_python/etl.py
```

Extract hanya menarik, tidak membersihkan

```
def extract_barat() -> pd.DataFrame:
    # dtype=str itu disengaja, sikapnya sama dengan kolom VARCHAR di
    # staging Sesi 1: jangan memaksakan tipe data sebelum tahu isinya bersih.
    return pd.read_csv(cfg.DATA / "penjualan_barat.csv", dtype=str)
```

`dtype=str` adalah titik temu dengan Sesi 1. Di SQL kita memakai kolom `VARCHAR` untuk semuanya; di pandas kita memakai `dtype=str`. Prinsipnya identik, cuma alatnya berbeda. Alasan nyatanya: satu baris dengan jumlah kosong akan menggagalkan seluruh pembacaan kalau pandas dipaksa menebak tipe angka.

Alasan kedua yang lebih penting di dunia kerja: makin cepat extract selesai, makin singkat sistem kasir terganggu. Pembersihan bisa dikerjakan belakangan, di mesin kita sendiri.

Transform mencatat setiap baris yang dibuang

```
catatan = []          # log: (langkah, masuk, dibuang, keluar)

def lapor(langkah, masuk, keluar):
    catatan.append((langkah, masuk, masuk - keluar, keluar))
```

Fungsi tiga baris ini adalah bagian paling berharga di seluruh sesi. Keluarannya:

Langkah	masuk	dibuang	keluar
Kasir: menyamakan bentuk kolom	389	0	389
Excel: membuang baris tanpa jumlah	188	5	183
Excel: membuang baris duplikat	183	12	171
Excel: mencocokkan nama produk	171	0	171
HASIL AKHIR			560

ETL yang hanya mencetak “selesai” tidak bisa dipercaya. Yang bisa dipercaya adalah ETL yang bisa ditanya: dari 188 baris, ke mana perginya 17? Tabel di atas menjawabnya baris per baris. Kalau kamu memakai ETL di skripsi, pertanyaan ini hampir pasti muncul di sidang, jadi siapkan lognya sejak awal.

Penyeragaman nama produk memakai cara yang sama dengan Sesi 1, bukan `.str.title()`:

```
df["kunci"] = df["nama_produk"].str.strip().str.upper()
produk["kunci"] = produk["nama_produk"].str.upper()
df = df.merge(produk[["kunci", "nama_produk", "kategori"]],
              on="kunci", how="left", suffixes=("_asli", ""))
tidak_cocok = int(df["nama_produk"].isna().sum())
if tidak_cocok:
    print(f"PERINGATAN: {tidak_cocok} baris tidak cocok ke tabel produk.")
```

Perhatikan `how="left"` lalu hitung yang `NaN`. Kalau memakai `how="inner"`, baris yang gagal dicocokkan hilang tanpa suara dan tidak ada yang tahu. Dengan `left` kita masih bisa menghitung dan memperingatkan. Ini pola yang berlaku di mana saja: **jangan pernah membuang baris tanpa menghitungnya lebih dulu.**

Load punya urutan yang tidak bisa ditawar

```
bersih.to_sql("penjualan_bersih_py", mesin_dw, if_exists="replace", index=False)
dimensi = muat_dimensi(mesin_dw, mesin_sumber, bersih)    # DIMENSI DULU
fakta    = muat_fakta(mesin_dw, bersih, dimensi)          # BARU FAKTA
buat_indeks(mesin_dw)                                    # INDEKS PALING AKHIR
```

Dimensi dulu, karena kunci dimensi baru ada setelah baris dimensinya terbentuk, sedangkan tabel fakta membutuhkan kunci itu. Membalik urutan ini akan menghasilkan tabel fakta penuh kunci kosong.

Setiap dimensi juga mendapat satu baris berkunci `-1`:

```
KUNCI_TIDAK_DIKETAHUI = -1
```

Itu sebabnya `dim_produk_py` berisi 9 baris (8 produk + 1 Tidak Diketahui) dan `dim_cabang_py` berisi 4. Perannya sama dengan baris “Tanpa Pelanggan” di Sesi 2: menyediakan tempat berlabuh supaya tidak ada baris fakta yang hilang hanya karena kuncinya tidak ketemu.

Indeks dibuat paling akhir, dan itu bukan kebetulan

```
"ALTER TABLE fact_penjualan_py "
"  ADD INDEX i_tanggal (tanggal_key), ADD INDEX i_produk (produk_key), "
"  ADD INDEX i_cabang (cabang_key),  ADD INDEX i_pelanggan (pelanggan_key)"
```

Tabel hasil `to_sql()` lahir tanpa indeks sama sekali. Tanpa langkah ini, query OLAP di Sesi 4 bisa berjalan berpuh kali lebih lambat — dan bukan dugaan: itu benar-benar terjadi saat modul ini diuji, query yang seharusnya selesai dalam sedetik tersangkut lebih dari dua menit.

Indeks selalu dibuat **sesudah** data dimuat, bukan sebelumnya, karena memelihara indeks sambil menyisipkan ribuan baris jauh lebih lambat daripada membangunnya sekali di akhir.

Titik periksa

Pemeriksaan	Nilai benar
fact_penjualan_py	560 baris
Omzet di tabel fakta	20.784.000
dim_tanggal_py	365
dim_produk_py / dim_cabang_py / dim_pelanggan_py	9 / 4 / 41
Gagal lookup tanggal, produk, cabang	0, 0, 0
Baris tanpa identitas pelanggan	167
Jumlah produk berbeda	8
Tanggal gagal dibaca	0

Dua angka pertama itu intinya seluruh sesi: **560 baris dan omzet 20.784.000**, sama persis dengan Sesi 1 yang **makai SQL**. Dua jalan yang sangat berbeda — satu lewat `INSERT ... SELECT`, satu lewat pandas — bertemu di angka yang identik.

Buktikan sendiri dengan satu query:

```
SELECT (SELECT SUM(total) FROM penjualan_bersih) AS versi_sql,  
       (SELECT SUM(total) FROM fact_penjualan_py) AS versi_python;
```

Kalau kedua kolom berbeda, salah satu jalur melewati sebuah aturan. Mencari tahu aturan mana yang terlewat adalah latihan debugging terbaik di seluruh praktikum ini.